

Amazon ECS Immersion Day

Amazon ECS and AWS Fargate

▶ Getting Started

▼ Fundamentals

Amazon ECS Cluster

Task definitions

Services

Updating a service

Enable ECS Exec

▶ Auto Scaling

▶ Networking

▶ Observability

▶ Security

▶ Automation

▶ Storage

Congratulations!

Amazon ECS Cluster

In this lab, you will build the core components of Amazon ECS, including an ECS cluster, task definition, and ECS service. The ultimate goal is to deploy a container sitting behind an Application Load Balancer (ALB).

Using the **VS Code terminal**, create an Amazon ECS Cluster named **retail-store-ecs-cluster** with [CloudWatch Container Insights](#) enabled. Container Insights collects, aggregates, and summarizes metrics and logs from your containerized applications and microservices:

 **Important**
To paste commands using Firefox browser, use **Shift+V** on Windows/Linux, **Command+Shift+V** on macOS

```
aws ecs create-cluster --cluster-name retail-store-ecs-cluster --region $AWS_REGION --settings name=containerInsights,value=enabled
```



You should receive the following output in the command line, where you can see that *CloudWatch Container Insights* is enabled:

```
1  {
2    "cluster": {
3      "clusterArn": "arn:aws:ecs:us-west-2:111111111111:cluster/retail-store-ecs-cluster",
4      "clusterName": "retail-store-ecs-cluster",
5      "status": "ACTIVE",
6      "registeredContainerInstancesCount": 0,
7      "runningTasksCount": 0,
8      "pendingTasksCount": 0,
9      "activeServicesCount": 0,
10     "statistics": [],
11     "tags": [],
12     "settings": [
13       {
14         "name": "containerInsights",
15         "value": "enabled"
16       }
17     ],
18     "capacityProviders": [],
19     "defaultCapacityProviderStrategy": []
20   }
21 }
```

To inspect the cluster you just created, navigate to the Amazon ECS Console:

Open Amazon ECS console 

Now that you have created the cluster, proceed to the next step to create an ECS Task Definition that you will use to deploy the UI service.

Previous

Next

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

Task definitions

A task definition is a blueprint that describes how a container (or containers) should run on Amazon ECS. It includes various configurations such as the container image to use, the required CPU and memory, the ports to open, and the environment variables needed.

Let's create the task definition to be used for the **UI Service**:

```
1  cat << EOF > retail-store-ecs-ui-taskdef.json
2  {
3      "family": "retail-store-ecs-ui",
4      "networkMode": "awsvpc",
5      "requiresCompatibilities": [
6          "FARGATE"
7      ],
8      "cpu": "1024",
9      "memory": "2048",
10     "runtimePlatform": {
11         "cpuArchitecture": "X86_64",
12         "operatingSystemFamily": "LINUX"
13     },
14     "containerDefinitions": [
15         {
16             "name": "application",
17             "image": "public.ecr.aws/aws-containers/retail-store-sample-ui:0.7.0",
18             "portMappings": [
19                 {
20                     "name": "application",
21                     "containerPort": 8080,
22                     "hostPort": 8080,
23                     "protocol": "tcp",
24                     "appProtocol": "http"
25                 }
26             ],
27             "essential": true,
28             "linuxParameters": {
29                 "initProcessEnabled": true
30             },
31             "healthCheck": {
32                 "command": [
33                     "CMD-SHELL",
34                     "curl -f http://localhost:8080/actuator/health || exit 1"
35                 ],
36                 "interval": 10,
37                 "timeout": 5,
38                 "retries": 3,
39                 "startPeriod": 60
40             },
41             "logConfiguration": {
42                 "logDriver": "awslogs",
43                 "options": {
44                     "awslogs-group": "retail-store-ecs-tasks",
45                     "awslogs-region": "$AWS_REGION",
46                     "awslogs-stream-prefix": "ui-service"
47                 }
48             }
49         }
50     ],
51     "executionRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskExecutionRole",
52     "taskRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskRole"
53 }
54 EOF
55
56 aws ecs register-task-definition --cli-input-json file:///retail-store-ecs-ui-taskdef.json
```

You can find detailed explanations of the parameters within the ECS task definition [in the official documentation](#). However, let's briefly discuss some key parameters:

Parameter	Description
family	A name for multiple versions of the task definition, specified with a revision number.
requiresCompatibilities	Determines where Amazon ECS launches the task or service based on the specified launch type.
taskRoleArn	An IAM role that allows the containers to call specified AWS APIs on your behalf.
executionRoleArn	An IAM role that grants the Amazon ECS container agent permission to make AWS API calls on your behalf.
networkMode	The Docker networking mode for the containers in the task. For ECS Fargate, this is restricted to the <code>awsvpc</code> mode.
cpu, memory	For Fargate, you need to specify values within predefined SKUs.
containerDefinitions	Detailed information such as the container image, port mappings, and health check settings for the container to be used in an ECS task.

You can retrieve the task definition using the AWS CLI:

```
aws ecs describe-task-definition --task-definition retail-store-ecs-ui
```

This command will display the full details of the task definition you just created.

Previous

Next

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

- Amazon ECS and AWS Fargate
- ▶ Getting Started
- ▼ Fundamentals
 - Amazon ECS Cluster
 - Task definitions
 - Services
 - Updating a service
 - Enable ECS Exec
- ▶ Auto Scaling
- ▶ Networking
- ▶ Observability
- ▶ Security
- ▶ Automation
- ▶ Storage
- Congratulations!

Services

An ECS service enables you to run and maintain a specified number of instances of a task definition simultaneously in an Amazon ECS cluster. If any of these tasks fail or stop for any reason, the ECS service scheduler launches another instance of your task definition to replace it, maintaining the desired number of tasks in the service. This ensures high availability for your application.

ECS services are used to manage long-running applications, microservices, or other software components that require high availability. Services in ECS can be integrated with Elastic Load Balancing (ELB) to distribute traffic evenly across the tasks in the service, providing a seamless way to deploy, manage, and scale your containerized applications.

For your convenience, we have pre-built resources related to the Application Load Balancer (ALB) including the listener, target group, and security group in advance.

Let's create the ECS service:

```
export UI_TARGET_GROUP_ARN=$(aws elbv2 describe-target-groups --names ui-application \
--query 'TargetGroups[0].TargetGroupArn' --output text)

aws ecs create-service \
  --cluster retail-store-ecs-cluster \
  --service-name ui \
  --task-definition retail-store-ecs-ui \
  --desired-count 2 \
  --launch-type FARGATE \
  --load-balancers targetGroupArn=${UI_TARGET_GROUP_ARN},containerName=application,containerPort=8080 \
  --network-configuration "awsVpcConfiguration={subnets=[${PRIVATE_SUBNET1},${PRIVATE_SUBNET2}],securityGroups=[${UI_SG_ID}],assignPublicIp=DISABLED}"
```

It may take several minutes for ECS to deploy the service and for it to register as stable. While this is happening, you can explore the service in the ECS console:

Open Amazon ECS console

You can also wait for the service to stabilize with the AWS CLI (~ 2 min):

```
aws ecs wait services-stable --cluster retail-store-ecs-cluster --services ui
```

Once the service is stable, you can view the running tasks from the CLI:

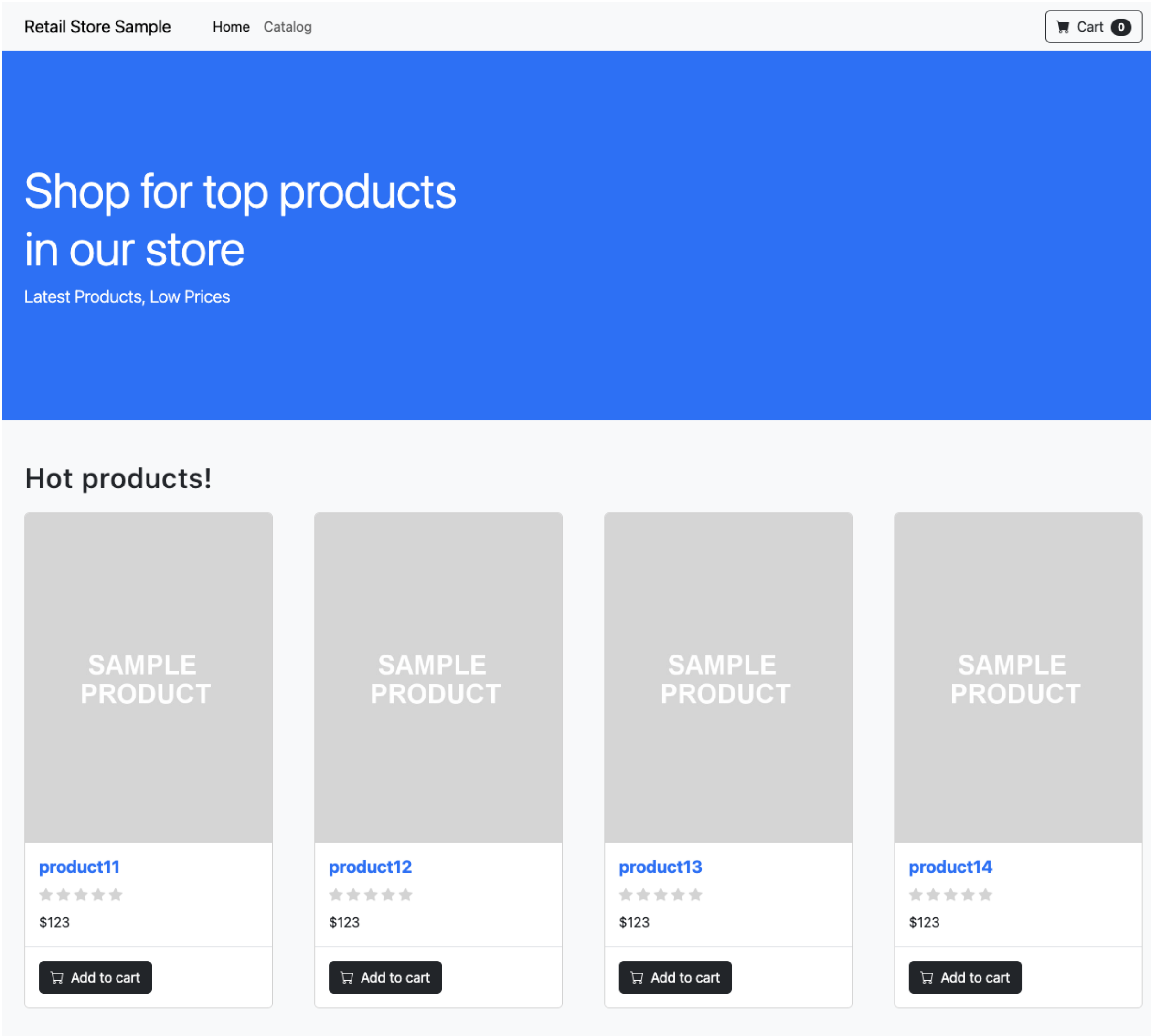
```
aws ecs describe-tasks \
  --cluster retail-store-ecs-cluster \
  --tasks $(aws ecs list-tasks --cluster retail-store-ecs-cluster --query 'taskArns[]' --output text) \
  --query "tasks[*].[group, launchType, lastStatus, healthStatus, taskArn]" --output table
```

You can retrieve the load balancer URL like so:

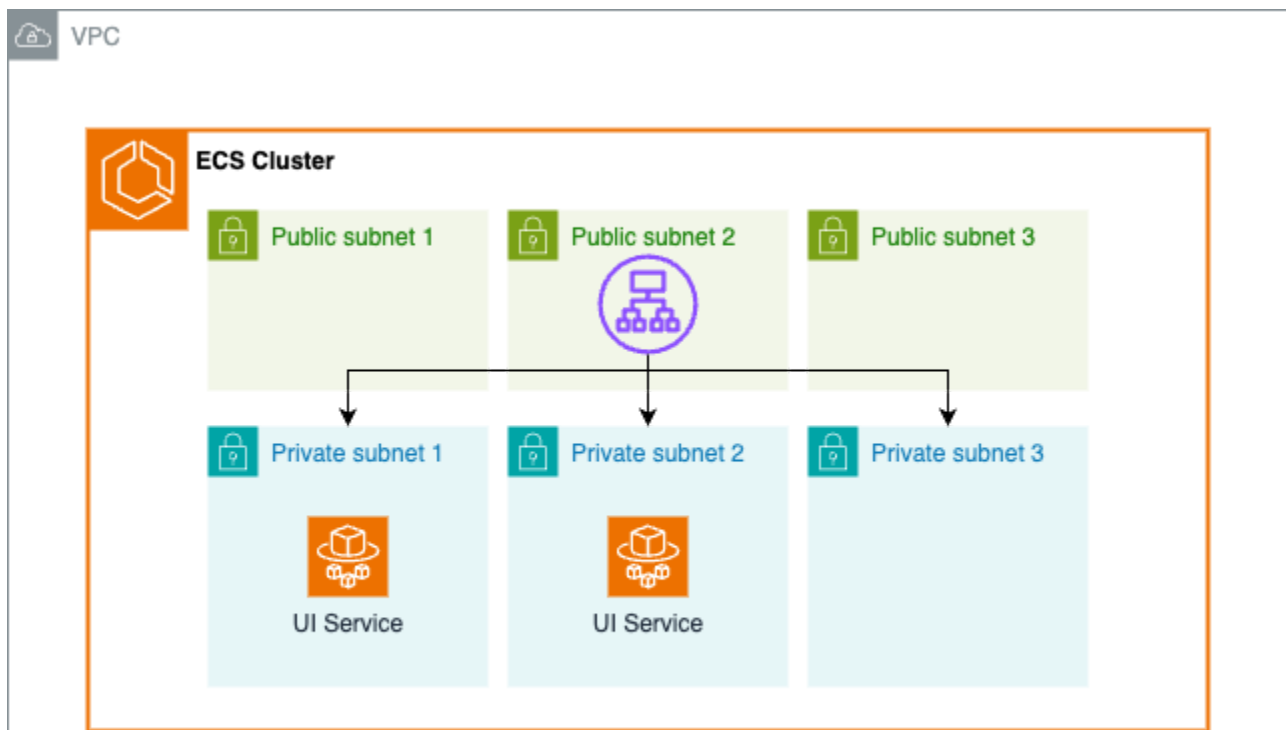
```
export RETAIL_ALB=$(aws elbv2 describe-load-balancers --name retail-store-ecs-ui \
--query 'LoadBalancers[0].DNSName' --output text)

echo http://${RETAIL_ALB} ; echo
```

Paste the URL into a web browser to access the application. You should see a screen like the one below.



After creating the fundamental components, the architecture of the services configured as of now is shown below.



Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose "Customize" or "Decline" to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose "Accept" or "Decline." To make more detailed choices, choose "Customize."

Accept

Decline

Customize

- Amazon ECS and AWS Fargate
 - ▶ Getting Started
 - ▼ Fundamentals
 - Amazon ECS Cluster
 - Task definitions
 - Services
 - Updating a service
 - Enable ECS Exec
 - ▶ Auto Scaling
 - ▶ Networking
 - ▶ Observability
 - ▶ Security
 - ▶ Automation
 - ▶ Storage
- Congratulations!

Updating a service

In this section, you'll learn how to update an ECS service. This process is useful in scenarios such as changing the container image or modifying the configuration.

Environment variables are one of the primary mechanisms used to configure container workloads, regardless of the orchestrator. You'll alter the configuration of the UI service by passing a new environment variable that will alter the behavior of the workload. In this case, you'll use the `RETAIL_UI_BANNER` setting, which will add a banner to the page.

Environment variables are expressed in ECS task definitions with a name and a value like so:

```
"environment": [  
  {  
    "name": "RETAIL_UI_BANNER",  
    "value": "We've updated the UI service!"  
  }  
]
```

To update the **UI task definition** with the environment variable:

```
1 cat << EOF > retail-store-ecs-ui-updated-taskdef.json  
2 {  
3   "family": "retail-store-ecs-ui",  
4   "executionRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskExecutionRole",  
5   "taskRoleArn": "arn:aws:iam::${ACCOUNT_ID}:role/retailStoreEcsTaskRole",  
6   "networkMode": "awsvpc",  
7   "requiresCompatibilities": [  
8     "FARGATE"  
9   ],  
10  "cpu": "1024",  
11  "memory": "2048",  
12  "runtimePlatform": {  
13    "cpuArchitecture": "X86_64",  
14    "operatingSystemFamily": "LINUX"  
15  },  
16  "containerDefinitions": [  
17    {  
18      "name": "application",  
19      "image": "public.ecr.aws/aws-containers/retail-store-sample-ui:0.7.0",  
20      "portMappings": [  
21        {  
22          "name": "application",  
23          "containerPort": 8080,  
24          "hostPort": 8080,  
25          "protocol": "tcp",  
26          "appProtocol": "http"  
27        }  
28      ],  
29      "essential": true,  
30      "linuxParameters": {  
31        "initProcessEnabled": true  
32      },  
33      "environment": [  
34        {  
35          "name": "RETAIL_UI_BANNER",  
36          "value": "We've updated the UI service!"  
37        }  
38      ],  
39      "healthCheck": {  
40        "command": [  
41          "CMD-SHELL",  
42          "curl -f http://localhost:8080/actuator/health || exit 1"  
43        ],  
44        "interval": 10,  
45        "timeout": 5,  
46        "retries": 3,  
47        "startPeriod": 60  
48      },  
49      "logConfiguration": {  
50        "logDriver": "awslogs",  
51        "options": {  
52          "awslogs-group": "retail-store-ecs-tasks",  
53          "awslogs-region": "${AWS_REGION}",  
54          "awslogs-stream-prefix": "ui-service"  
55        }  
56      }  
57    }  
58  ]  
59 }  
60 EOF
```

Now, use the `register-task-definition` command to update the task definition:

```
aws ecs register-task-definition --cli-input-json file://retail-store-ecs-ui-updated-taskdef.json
```

It's important to note that ECS task definitions are immutable, which means they cannot be modified after creation. Instead, the above command will create a new task definition revision, which is a copy of the current task definition with the new parameter values replacing the existing ones.

You can check that you now have multiple task definition revisions with the following command:

```
aws ecs list-task-definitions --family-prefix retail-store-ecs-ui
```

The output will show all of the revisions for the UI task definition family:

```
{  
  "taskDefinitionArns": [  
    "arn:aws:ecs:us-west-2:XXXXXXXXXX:task-definition/retail-store-ecs-ui:1",  
    "arn:aws:ecs:us-west-2:XXXXXXXXXX:task-definition/retail-store-ecs-ui:2"  
  ]  
}
```

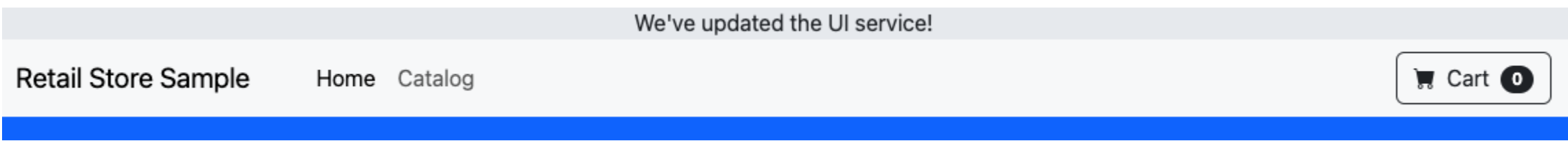
Now you need to update the ECS service to use the new task definition revision:

```
aws ecs update-service --cluster retail-store-ecs-cluster --service ui --task-definition retail-store-ecs-ui
```

Wait for ECS to deploy the changes to the service (~ 5 min):

```
aws ecs wait services-stable --cluster retail-store-ecs-cluster --services ui
```

Now refresh your browser, and you will see the banner has been added based on your environment variables:



Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

Amazon ECS Immersion Day

- Amazon ECS and AWS Fargate
 - ▶ Getting Started
 - ▼ Fundamentals
 - Amazon ECS Cluster
 - Task definitions
 - Services
 - Updating a service
 - Enable ECS Exec**
 - ▶ Auto Scaling
 - ▶ Networking
 - ▶ Observability
 - ▶ Security
 - ▶ Automation
 - ▶ Storage
 - Congratulations!

[Amazon ECS Immersion Day](#) > [Fundamentals](#) > Enable ECS Exec

Enable ECS Exec

In this section, you'll enable the **ECS Exec** feature to run commands in or get a shell to a container running on an EC2 instance or Fargate. Enabling ECS Exec is beneficial for operational management and advantageous from a security perspective. It offers controlled access to containers running in your ECS tasks, allowing for secure, audited, and interactive troubleshooting without having to SSH into hosts.

By leveraging IAM policies and roles, you can tightly control who has access to execute commands within containers, thus enhancing the overall security posture. Additionally, all commands executed through ECS Exec are logged in CloudWatch, providing an audit trail for compliance and monitoring purposes. [More information can be found here](#) .

Follow these steps to enable and use ECS Exec:

1. Set the IAM role for the user
2. Set the IAM role for the ECS task role
3. Set up the environment
4. Enable Amazon ECS Exec on the service
5. Connect to the ECS task

Set the IAM role for the user

Since you'll be using ECS Exec from your IDE, ensure that the IAM role attached to the IDE has the necessary IAM policies. Update the IAM role associated with the EC2 instance running your IDE by adding the following in-line policy:

```
cat << EOF > ecs-exec-command-policy.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ecs:ExecuteCommand",
        "ecs:DescribeTasks"
      ],
      "Resource": [
        "arn:aws:ecs:${AWS_REGION}:${ACCOUNT_ID}:task/retail-store-ecs-cluster/*",
        "arn:aws:ecs:${AWS_REGION}:${ACCOUNT_ID}:cluster/*"
      ]
    }
  ]
}
```

```
aws iam put-role-policy \
  --role-name $(aws sts get-caller-identity --query 'Arn' | cut -d '/' -f2) \
  --policy-name AmazonECSExecCommand \
  --policy-document file://ecs-exec-command-policy.json
```

▶ Expand to check if the policy is correctly configured.

 In this lab, you've only added the most basic IAM policy. Additional IAM Policies may be required to utilize various ECS Exec features such as CloudWatch logs.

Set the IAM role for the ECS Task Role

ECS Exec needs a task IAM role for SSM communication. We've pre-created this IAM role for the **UI Service** task. Let's check its permissions:

```
aws iam get-role-policy \
  --role-name retailStoreEcsTaskRole \
  --policy-name $(aws iam list-role-policies --role-name retailStoreEcsTaskRole --query 'PolicyNames[0]' --output text)
```

The result should look like this:

```
{
  "RoleName": "retailStoreEcsTaskRole",
  "PolicyName": "RetailStoreEcsTaskRoleDefaultPolicy3AF87919",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Action": [
          "ssmmessages:CreateControlChannel",
          "ssmmessages:CreateDataChannel",
          "ssmmessages:OpenControlChannel",
          "ssmmessages:OpenDataChannel"
        ],
        "Resource": "*",
        "Effect": "Allow"
      }
    ]
  }
}
```

Set up the environment

You need the AWS CLI and the [Session Manager plugin for the AWS CLI](#) . Both of these utilities have already been installed in the IDE for this workshop. Please review the linked documentation to set this up in your own environment.

Enable Amazon ECS Exec on the task

Update the **UI Service** to enable Amazon ECS Exec using the `--enable-execute-command` flag:

```
aws ecs update-service \
  --cluster retail-store-ecs-cluster \
  --service ui \
  --task-definition retail-store-ecs-ui \
  --enable-execute-command \
  --force-new-deployment
```

Wait for ECS to deploy the changes to the service (~ 5 min):

```
aws ecs wait services-stable --cluster retail-store-ecs-cluster --services ui
```

Run the following command to select one of the running **UI tasks** with `enableExecuteCommand` enabled:

```
ECS_EXEC_TASK_ARN=$(aws ecs list-tasks --cluster retail-store-ecs-cluster \
  --service-name ui --query 'taskArns[]' --output text | \
  xargs -n1 aws ecs describe-tasks --cluster retail-store-ecs-cluster --tasks | \
  jq -r '.tasks[] | select(.enableExecuteCommand == true) | .taskArn' | \
  head -n 1)
```

Check the result using the `echo` command:

```
echo $ECS_EXEC_TASK_ARN
```

Connect to the ECS Task

Start your `/bin/bash` interactive session in the running task:

```
if [ -z ${ECS_EXEC_TASK_ARN} ]; then echo "ECS_EXEC_TASK_ARN is not correctly configured!"; else
aws ecs execute-command --cluster retail-store-ecs-cluster \
  --task $ECS_EXEC_TASK_ARN \
  --container application \
  --interactive \
  --command "/bin/bash"
fi
```

You should see output like this:

```
The Session Manager plugin was installed successfully. Use the AWS CLI to start a session.

Starting session with SessionId: ecs-execute-command-vvdysulqbcz2txr2d262sw2s64
bash-5.2#
```

Within the new session, you can execute the following commands:

```
cat /etc/os-release

df -h
```

To terminate your session:

```
exit+
```

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose "Customize" or "Decline" to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose "Accept" or "Decline." To make more detailed choices, choose "Customize."

Accept

Decline

Customize